ODSI
Office of Data Science and Informatics

SNWG · HLS

Finding Efficiencies in HLS Production

The hard lessons six years of operating HLS have taught us about cost and risk and efficiency and how other solutions can leverage these lessons.

Sean Harkins · Development Seed / ODSI

ORIENTATION

HLS at a glance

2021

Global v2.0 production running continuously since

~15,000

Landsat and Sentinel-2 granules processed a day

~$300k

AWS processing costs in 2025, not including S3 storage

Harmonized Landsat and Sentinel-2 turns two missions into **one consistent 30 m surface reflectance record**, distributed by LP DAAC.

HLS is also an input. **Several other SNWG solutions build on top of it** so any improvements we make in latency and stability are shared with other products.

BACKGROUND

Why now?

5 Five years of operations

- Cloud architecture patterns have changed a lot since 2021
- We've learned the tough lessons about scaling and inefficiency the hard way
- Managing and integrating science code has been a continuous challenge

A new requirement

- The HLS low-latency solution requires a new auxiliary data source and infrastructure with zero latency
- Simply patching our legacy libraries and systems to handle these requirements would have been costly and difficult to maintain
- We used these new requirements as a push to **re-architect** with the lessons we've learned

OUR GOALS

What we set out to improve

0

Low latency

The push: meet the new HLS low-latency solution requirements.

1

Reduce science code risk

Own, maintain and test the core algorithmic code that HLS relies on.

2

Reduce infrastructure costs to an absolute minimum

Refactor the bottlenecks and inefficiencies we've found.

4

Make observability central

Provide a complete picture of system state and health at all times.

2025 Infrastructure Spending



Of the $450k of AWS infrastructure cost in 2025. LaSRC modernization targets EC2 compute and the orchestration modernization targets the "other" and unlocks cheaper reprocessing.

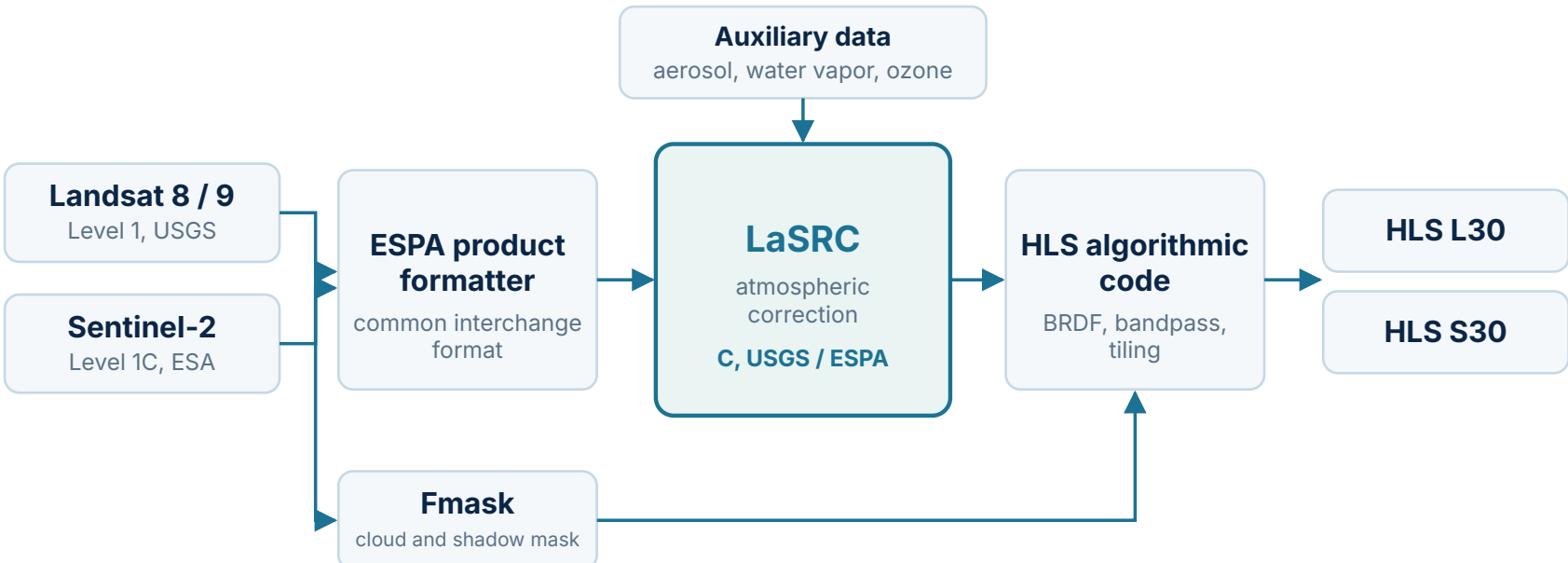
PART ONE

Modernizing LaSRC

Making core science code maintainable, performant and stable.

ORIENTATION

Where LaSRC sits in HLS



The product formatter, LaSRC and FMask are maintained outside our team. **LaSRC is the scientific heart of the product.**

2020

Why we chose ESPA

When we started HLS global processing we needed the LaSRC implementation that was **simplest to compile, containerize and maintain**. USGS ESPA was clearly that, and it brought more than just the algorithm.

❶ Build infrastructure

A working recipe for compiling the library into an image.

❷ Format conversion

The product formatter, turning Landsat and Sentinel distributions into one interchange format.

❸ Auxiliary data tooling

Utilities for obtaining and generating the water vapor and aerosol datasets LaSRC depends on.

At the time this was the best choice.

SCIENCE != PRODUCTION

Science code is not production code

Science code optimizes for

- Answering a research question correctly
- Rapid iteration on the algorithm itself
- Being run by the person who wrote it
- Publishing a result, then moving on
- Flexibility to try something different tomorrow



A production system requires

- Reproducibility across millions of granules
- Change isolated, reviewable and reversible
- Being operated and maintained by people who did not write it
- Operating continuously for a decade
- Stability, with change under explicit control

SCIENCE != PRODUCTION

The ESPA LaSRC C Codebase

A One maintainer

A single engineer mainly responsible for the entire codebase. This is normal for science code but is a risk factor for an operational product.

B Stuck on legacy Python

The very complex compilation chain provided for the ESPA base container made upgrading Python versions difficult

C Deprecated build dependencies

Compilation relies on several dependencies that are no longer actively maintained, so the build gets harder to reproduce every year.

D No public repository or licence

Nowhere to file an issue, track a fix, or contribute back. Our questions and our patches have no route in.

E Adding GDAS auxiliary data is hard and costly

The low latency solution needs NOAA GDAS auxiliary data, but VIIRS and MODIS are separate code paths that cannot be mixed. Adding a third source means very large changes that are difficult to track and test, in code we do not own.

No regular release cycle, **no tagged versions to pin**, no performance regression testing, no baseline testing against reference images.

THE REQUIREMENT

What makes a system "production" grade?

1. **Automated fidelity testing.** Every change validated against a suite of test granules, proving no reflectance or performance regression.
2. **Automated unit testing.** Structural and semantic correctness checked on every commit, not inferred from the output looking reasonable.
3. **Isolated, reviewable change.** An algorithmic update arrives as a well structured pull request we can selectively test, apply and roll back.
4. **Automated build and deploy.** Any update compiles and deploys with minimal change to our continuous integration pipelines.
5. **Pinned, tagged releases.** Production deployments reference a specific version, and that version is recorded in product metadata.

And underneath all five: a codebase **flexible enough to incorporate upstream format changes and new features**, balancing fidelity against compute cost, in a language new engineers will want to work in.

OUR VISION

A community driven atmospheric correction model

Even with these proposed updates, we would still depend on a legacy codebase maintained by a very small team. What we actually want is a library with **a larger community of maintainers**, led by a consortium of organisations, that can react quickly to upstream change from ESA and NASA.

1 A Rust port

Rewrite the ESPA LaSRC C library in Rust: equal or better numerical performance, with language level safety and maintainability.

2 A reference image archive

A curated set of Landsat and Sentinel granules for reflectance target assessment and for memory and CPU profiling over time.

3 Automated comparison notebooks

A repository of fully automated notebooks that compare and analyze LaSRC outputs, so inter-comparison testing is a pipeline step, not a person assessing outputs.

The goal is not a better fork. It is **a codebase other organizations want to both use and contribute to**.

THE CHOICE

Why Rust



1 Easy Python integration

Equal,throughput than the C implementation.

2 Safety at the language level

Memory and type safety are enforced by the compiler rather than by reviewer discipline, which is exactly where a long lived scientific codebase tends to erode.

3 A larger pool of future maintainers

Rust has real momentum in academic and engineering communities. The contributor pool for a Rust project is bigger than for a large legacy C codebase.

The third reason is the one that actually drove the decision. **Community collaboration is a core goal** for any healthy software project.

THE OBSTACLE

So why had we not done it?

5 to 6

months of **1 FTE** was my initial estimate for building and testing a Rust port

For a team of our size, running a production system while piloting other SNWG solutions, that was **not a block of time we could prioritize.**

The HLS low-latency solution changed this prioritization

As we scoped the ESPA changes needed to add alternative auxiliary data support for the low latency requirement, we realized it might be **less work to add that support in a new, simpler Rust codebase** than to retrofit it into the existing one.

TIMING

Agents to the rescue

This prioritization coincided with a steep rise in the coding capability of large language models and the adoption of **multi-agent harnesses** that decompose an engineering task into work streams which can run in parallel.

1 Parallel work streams

Several agent sessions working concurrently on different portions of the port rather than one engineer working sequentially.

2 A reference to aim at

The model was initially prompted to replicate the ESPA LaSRC codebase as closely as possible, with the C source as the specification.

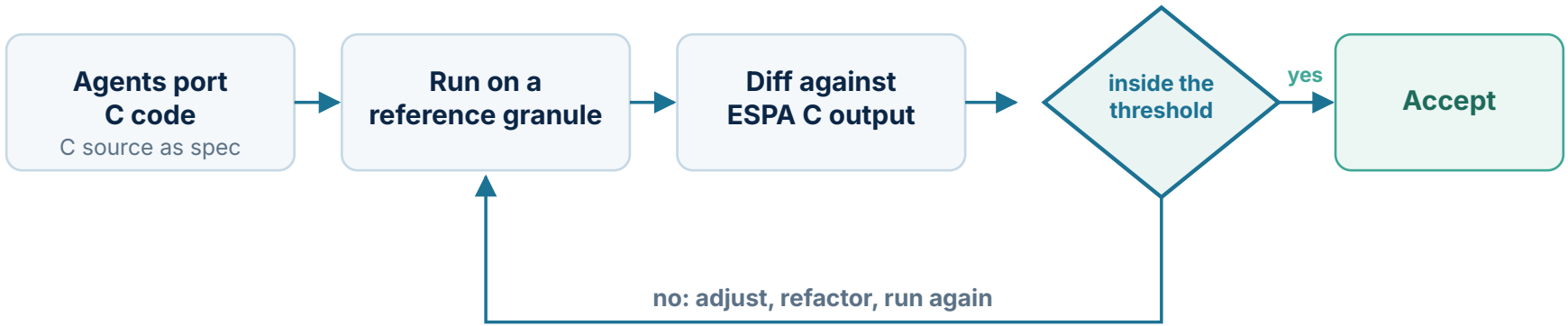
3 An objective stopping condition

Output compared continuously against a reference ESPA LaSRC granule, refactoring until the delta was less than the sensor noise floor.

The critical ingredient was not the model. It was **having a reference output for the model to measure against**.

APPROACH

The loop



Agents supplied the code. **The reference output supplied the source of truth.** Neither works without the other.

A GOOD START

A strong foundation, not a finished port

What we had

- A complete Rust implementation of the algorithm
- Drastically improved Python wrapper and CLI code
- Good agreement on the initial test granules
- Months of estimated effort compressed into a few days

What we did not have

- Evidence across a representative range of targets
- Any way to distinguish agreement from coincidence
- Confidence that alternative implementation choices were deliberate

VALIDATION

The testing suite we have always needed

Since the start of the HLS project, the production team has asked for a large scale automated validation suite, so that algorithmic changes could be validated safely while measuring the effect across a wide, representative set of targets.

1 Built in house

Rather than depend on an external team for our validation testing, **Trang Vo and Madhu Sridhar** have been building and refining this system continuously.

2 Fidelity and cost together

Resource utilization can be logged while outputs are generated, so the same run that proves reflectance agreement also measures **performance regression**.

hls-application · `https://github.com/NASA-IMPACT/hls-application/blob/lasrc_validation/notebooks/LaSRC_container_validation.ipynb`

VALIDATION

What the suite actually does

353

granules compared, each at the immediate LaSRC stage and the final HLS stage

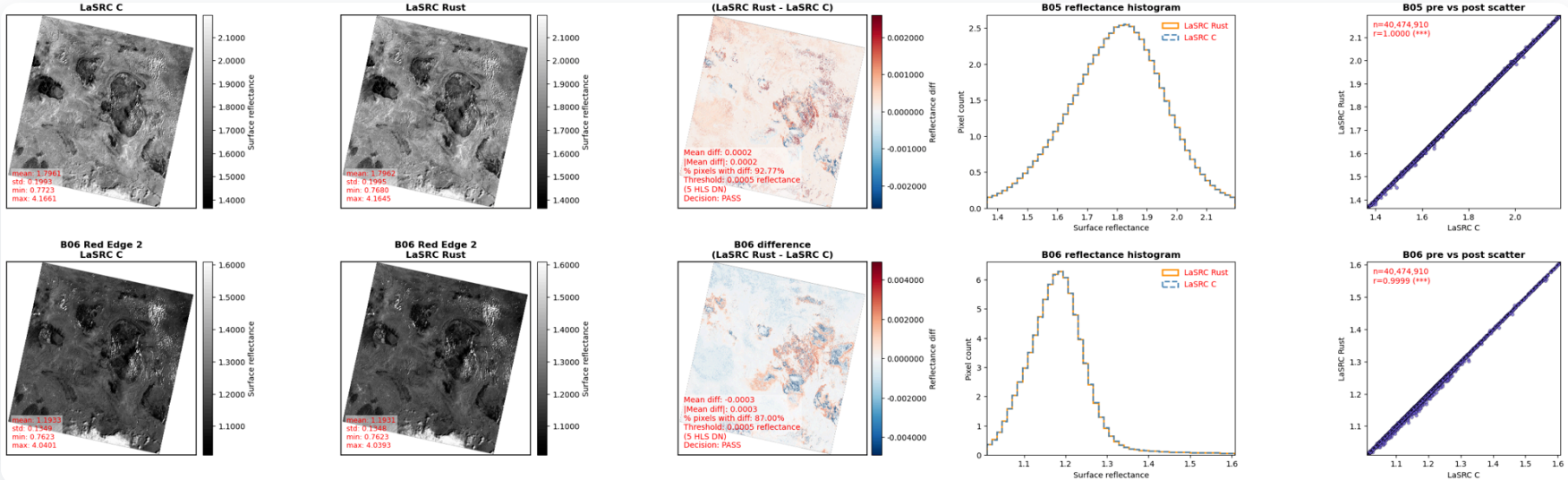
7

bands compared per granule, B01 Coastal Aerosol through B07 SWIR2

1. **Discover and pair.** List rasters under both output prefixes, detect stage, keep only symmetric pairs so nothing is compared against a gap.
2. **Compare per pixel.** Decode reflectance for both versions, compute the difference field, and summarize it.
3. **Decide against a threshold.** A band passes when the absolute mean reflectance difference is within the configured threshold for that band.

VALIDATION

What one comparison looks like



Two of seven band rows, one granule. Left to right: C reflectance · Rust reflectance · difference with decision · histogram overlay · scatter

REALITY

Humans in the loop

When we ran the Rust code and validation suite against the full set of test granules, we **discovered a host of bugs**.

The foundation had gaps

Agreement on a handful of granules had concealed a range of inconsistencies that only appeared across a wider set of granules and targets.

The review loop

Chris Holden tackled these incrementally, combining agent driven refactoring with manual review. There was a lot to address, but working in an agentic loop accelerated it dramatically.

While LLM usage accelerated this process it is only as capable as the engineer / scientist working with it.

BUGS / INCONSITENCIES

Four main classes of issues

1 Type conversion, faithfully replicated

The ESPA C codebase is inconsistent about type conversion in a number of places. The model replicated many of those conversions faithfully, which produced **residual accumulation differences**.

2 Invalid auxiliary retrieval

Where auxiliary retrieval fails, the model decieded to set transmittance to **zero**. The C implementation sets it to the **closest non-fill neighbour**.

3 Scene center geometry

The model chose to **calculate** the scene center geometry. The C version takes solar zenith from metadata and sets view zenith and relative azimuth to zero.

4 Scene center interpolation

For center pressure, ozone and water vapor the model chose **bilinear interpolation** where C used **nearest neighbour**.

MODELS ARE CAPABLE

Sometimes the model's choices were more correct than the ESPA codebase

What we chose to do

- Match the C implementation exactly, including where it is inconsistent
- Accept a known worse choice in order to prove equivalence first
- Record each divergence to address later

Why that was the right order

- Equivalence is the only claim that can be demonstrated objectively
- Once demonstrated, every divergence can be addressed **individually**
- Each becomes a reviewable change with a measurable effect, not an argument

Calculating the scene geometry is arguably **more accurate** than reading one angle from metadata and zeroing the other two. We matched C anyway, on purpose.

RESULTS

Where we are now

100%

Alignment between the Rust and ESPA C outputs, within the **5 DN** threshold, across all **353** granules in the reference set

30%

Faster than the ESPA C implementation

7%

Less memory used

Why 5 DN

The threshold was chosen on feedback from the science team, so that alignment between the ESPA and Rust outputs is measured against **a defined target rather than a judgement call**. 5 HLS DN corresponds to 0.0005 reflectance.

Why this number matters more than it looks

These are the gains from **targeted tweaks only**, made while the priority was matching C exactly. Almost every structural performance opportunity is still on the table, and at global scale runtime is directly processing cost.

NEXT STEPS

What is next, in priority order

1. **GDAS auxiliary data.** The largest rationale for undertaking the port was to reduce the effort of switching to GDAS based auxiliary data for the HLS low-latency solution. We begin testing and evaluating that code path in this PI.
2. **Structural refactoring.** The Rust port currently mostly mirrors the original ESPA C structure. We will incrementally restructure it to improve maintainability and to make it easier for outside contributors to work on.
3. **Performance optimization.** Beyond the targeted tweaks already made, there is a large space of changes available to reduce runtime, resource utilization and overall processing cost.
4. **Collaborative HLS Science + Production team evaluation** We'll review the results of all validation suite output with the HLS science team.
5. **Production deployment** After the finalized Rust port has gotten approval from the science team we'll integrate it into production.

For all these steps, the time to implement and test is **drastically lower than the same work would take in the ESPA codebase.**

PART TWO

Next-generation infrastructure

Converting lessons into efficiency

TODAY

Inefficiency in our current system

Step Functions
Use of repeated polling adds many costly state transitions

→

AWS Batch
One long job per granule: fmask, LaSRC, post-processing, upload

→

RDS Log Database
Always running despite spiky load

1 Granules silently dropped during backfill

At massive traffic loads database timeouts caused logging failures leaving granule status untracked.

2 Polling cost and rate limits

About 113 billed state transitions per granule, and AWS API limits at 10 to 20 thousand concurrent jobs.

3 Spot interruption risk for long jobs

LaSRC is about 55% of Sentinel-2 compute and a third of Landsat's. An interruption late in a job throws away the most expensive work.

4 History pruned away

The audit trail that could be leveraged for reprocessing and reconciliation is deleted to keep the database performant.

THE NEW DESIGN

Fully event-driven with S3 as the log database



Ancillary Data Arrival: When new ancillary data arrives an S3 key search identifies all the granules awaiting processing.

Spot Interruption: retry queue, resubmitted automatically

Unexpected Failure: dead-letter queue, redriven after a fix

The core principle - **S3 is the event coordination mechanism and the log database.** This means no infrastructure runs when events aren't happening.

COST

Orchestration cost, line by line

Annual cost	Current	New (fully event-driven)
Step Functions	$15,518	not used
RDS database	$4,375	not used
Lambda	$205	~$2
S3 ancillary data checks	~$60	~$0
S3 event store	not used	~$180
SQS and EventBridge	not used	~$15
Total	**~$20,160**	**~$200**

~100×

cheaper to orchestrate the same 15,000 granules a day.

2025 bill: Step Functions plus RDS came to about $20.7k which is in line with the model projections.

IN DETAIL

Paying for polling



A 20 to 30 minute job is polled every 30 to 60 seconds. Every poll is a **wait, a Lambda call and a decision**, and each one is billed as a state transition.

Polling is **about $15,518 a year** in Step Functions alone. An event costs nothing until something actually happens.

REPROCESSING

A full reprocessing campaign

Orchestration cost to reprocess the full archive **~30M granules** (for example, HLS v3):



From **$55–85k** to **about $4.1k** with only SPOT market availability as a limit on how quickly the campaign can be completed.

OBSERVABILITY

Questions we can now answer

? Internal reconciliation - Did every product reach LP DAAC?

We will now have a complete processing log we can use for reconciliation.

? External reconciliation - Did we attempt every granule ESA and USGS published?

Compare our full input log against their inventories.

? Full history - What happened on each processing attempt?

Our current log system only records the most recent attempt and failure.

The full history is kept **indefinitely, for about $180 (increasing slightly annually) a year**. No database to prune, no extra service to run.

ROLLOUT

Tested on new solutions.

Already proven

The HLS-VI historical backfill processed **~30M granules** using this event log pattern.

The log never needed pruning, retries handled Spot interruptions cleanly, and weekly progress reports were generated with a single query.

0. **Phase 0, Shadow.** The new job monitor watches the current system read-only. No changes to existing system but valid log information is produced.
1. **Phase 1, Fully event-driven.** Retire Step Functions polling and the database.
2. **Phase 2, Split jobs.** Fmask first so that high cloud granules exit early reducing full SPOT interruptions
3. **Phase 3, Reconcile.** Build new internal and external reconciliation systems from the structured logs.

REUSE

The BDEC Monthly Composites Use The Same Infrastructure

HLS orchestration has two parts: **submitting units of work** and **logging job outcomes**. We built the logging portion, improved it and reused it.

HLS-VI
Utilized the next generation watch AWS Batch job state change events

→

HLS-VI historical backfill
Incrementally improved the design based on large scale use.

→

Packaged the logging components as a reusable library

→

HLS monthly composites (BDEC)
Reused the library and the VI backfill's submission approach

→

HLS production, in shadow
Proving scale, cost and data collection without disrupting the current system

Iterating across HLS-related SNWG solutions **solved half the puzzle of next-generation HLS orchestration** while simultaneously reducing the work needed to produce derived HLS products.

PART THREE

Applying these lessons to other SNWG solutions

How can we generalize what we've learned producing HLS, HLS-VI and HLS BDEC Composites to support the rest of the SNWG ecosystem.

ACROSS SOLUTIONS

Patterns we see

WHAT WE SEE		WHAT WE LEARNED
Whole granules copied to local disk, sometimes more than once (sometimes this is the most optimal pattern)	→	**Read only the bytes you need**, straight from cloud-optimized sources
Loss of work due to SPOT interruption of multi-step jobs	→	**Split jobs** so a Spot interruption costs minutes, not hours
Always running servers and databases, paid for while idle	→	**Use pure event-driven and scale to zero**: pay only when something arrives
Legacy Fortran or C wrapped in layers of Python	→	**Build a robust validation suite and port with agents**
Homemade audit scripts to check what was produced	→	**An append-only log** you can query
Reprocessing with no cost estimate up front	→	**Model the cost** before you run the campaign

SELF-ASSESSMENT

Questions to ask about your own system

- [ ] Do you know your **average infrastructure cost per granule** and what a full archive reprocessing would cost?
- [ ] Are you paying for anything idle **while nothing is processing**?
- [ ] Do you rely on polling less stable systems like **CMR** rather than relying on events?
- [ ] Would a **Spot interruption** waste an hour of work?
- [ ] If a granule failed to process last month, could you **determine this with a single query**?
- [ ] Could you update your science code tomorrow and **validate that the output was the same**?

If you answered "no" or "not sure" to any of these, **we have been there**, and we would be excited to compare notes and lessons learned.

MULTIPLIER

HLS gains carry downstream

OPERA

DSWx-HLS surface water and DIST-HLS disturbance products are built from HLS.

FIRMS-2G

Burned area mapping uses the HLS vegetation indices.

HLS-LST

Land surface temperature builds on the HLS record.

When HLS gets **faster, cheaper and more reliable** all downstream products benefit. Lower HLS latency is lower latency for them.

Newer solutions, such as **GABAN** and **MWOW**, can leverage these patterns in **from the start** rather than retrofit them later as we had to.

TAKEAWAYS

1. **Measure first**
   Determine your average cost per granule. You cannot reduce what you have not measured.

2. **Pay for events, not for polling**
   For HLS, orchestration infrastructure cost is modeled to fall from about $20,160 to about $200 a year, and reprocessing from $55–85k to about $4.1k.

3. **Legacy science code is now affordable to modernize**
   AI agents made an intractable problem practical, but only because a validation suite proved the results.

4. **Talk to us**
   We want to share our designs, cost models and validation tooling so all SNWG solution teams can leverage them.